

Software Development:

- Software Development = Process of designing, building, testing, and maintaining software applications.
- It follows a structured approach to ensure quality, cost-effectiveness, and timely delivery.
- In practice, this is managed using Software Development Life Cycle (SDLC).

Phases of the Software Development Life Cycle:

1. Requirement Analysis

- Requirement gathering techniques (Interviews, Surveys, Prototypes)
- Functional vs Non-functional requirements
- Feasibility study (technical, operational, economic)

2. System Design

- High-level design (architecture, database design, data flow diagrams)
- Low-level design (module specifications, algorithms)
- Tools (UML diagrams, ER diagrams)

3. Implementation (Coding)

- Choice of programming languages
- Coding standards & guidelines
- Code reviews and version control

4. Testing

- Types of testing: Unit, Integration, System, Acceptance
- Black-box vs White-box testing
- Automation testing tools

5. Deployment

- Deployment strategies (Big bang, Incremental, Parallel, Phased)
- Continuous Deployment & DevOps

6. Maintenance

- Corrective, Adaptive, Perfective, Preventive maintenance
- Software evolution (Lehman's laws)

Software Development Life Cycle (SDLC) Models

SDLC Models describe how the different phases of software development are carried out.

1. Waterfall Model:

- Oldest and simplest model.
- Each phase flows into the next like a waterfall.

Process:

Requirement → Design → Implementation → Testing → Deployment → Maintenance

Advantages:

- Easy to understand and manage.
- Best for small projects with well-defined requirements.
- Clear milestones and documentation.

Disadvantages:

- Inflexible: once a phase is completed, hard to go back.
- Not suitable for projects where requirements change frequently.
- Late discovery of issues (during testing).

Use Case:

- Payroll system, banking software, government projects.

2. Iterative Model

- Develop software **in small iterations (cycles)**.
- Each iteration adds more features until the full system is ready.

Advantages:

- Early delivery of partial working system.
- Easier to fix problems in earlier iterations.
- Suitable for projects where requirements are not fully clear.

Disadvantages:

- Requires good planning.
- Cost may increase due to repeated cycles.

Use Case:

- Large systems where user feedback is required at each stage.

3. Incremental Model

- Similar to iterative model, but software is built **piece by piece** (increments).
- Each increment = fully working software module.

Advantages:

- Users get useful parts of the system earlier.
- Easier testing and debugging.
- Flexible and less risky.

Disadvantages:

- Needs proper design from the beginning.
- Final integration may be complex.

Use Case:

- E-commerce websites, online booking systems.

4. V-Model (Verification and Validation Model)

- Extension of Waterfall.
- Every development phase has a **corresponding testing phase**.

Structure:

- Left side: Requirement → Design → Coding
- Right side: Testing (Unit test, Integration test, System test, Acceptance test)

Advantages:

- Strong focus on testing.
- Defects can be caught early.

Disadvantages:

- Same rigidity as Waterfall.
- Costly if requirements change.

Use Case:

- Safety-critical systems (medical devices, defense software).

5. Spiral Model

- Combines **Iterative Model + Risk Analysis**.
- Development is done in spirals (loops), each loop has 4 steps:
 1. Planning
 2. Risk Analysis
 3. Engineering (development + testing)
 4. Evaluation

Advantages:

- Good for large, complex, and high-risk projects.
- Risk is managed at every stage.
- Flexible to requirement changes.

Disadvantages:

- Expensive and complex.
- Requires skilled risk management.

Use Case:

- Aerospace software, banking systems, large defense projects.

6. Big Bang Model

- No structured process. Developers start coding immediately.

Advantages:

- Very simple, less planning required.
- Best for small projects or academic purposes.

Disadvantages:

- Very risky, no documentation.
- Not suitable for large projects.

Use Case:

- Small experimental projects, student projects.

7. Agile Model

- Modern, **iterative and incremental** model.
- Focus on **customer collaboration, adaptive planning, and continuous delivery**.
- Work is divided into short cycles called **sprints** (1–4 weeks).

Principles:

- Individuals and interactions over processes.
- Working software over documentation.
- Customer collaboration over contract negotiation.
- Responding to change over following a plan.

Advantages:

- Highly flexible, customer feedback integrated quickly.
- Early and continuous delivery of working software.
- Best for dynamic projects.

Disadvantages:

- Requires experienced team.
- Customer must be available throughout.
- Less focus on documentation.

Use Case:

- Mobile apps, web applications, startups, rapidly changing environments.

8. Prototype Model

- A **working prototype** (sample) of software is built first.
- Users give feedback, then final system is developed.

Advantages:

- Helps users understand system requirements.
- Reduces risk of wrong requirements.

Disadvantages:

- Prototypes may cause misunderstanding (users may think it's the final product).
- Time consuming.

Use Case:

- Projects where requirements are unclear (e.g., new product ideas).

9. RAD (Rapid Application Development) Model

- Based on **prototyping and iterative delivery**.
- Emphasis on quick development using tools, code reusability, and automation.

Advantages:

- Faster delivery.
- Encourages user involvement.

Disadvantages:

- Not suitable for large, complex systems.
- Requires skilled developers and tools.

Use Case:

- Small-to-medium projects with short deadlines.

Capability Maturity Model (CMM)

The **Capability Maturity Model (CMM)** was developed by **SEI (Software Engineering Institute, Carnegie Mellon University)** to assess and improve the **software process maturity** of organizations.

5 Levels of CMM Maturity

1. Level 1 – Initial (Chaotic)

- Process is ad-hoc, unpredictable.
- Success depends on individual effort, not organization.
- No defined standards.

2. Level 2 – Repeatable

- Basic project management practices are established.
- Past successes can be repeated.
- Example: Cost estimation, schedule tracking.

3. Level 3 – Defined

- Processes are documented and standardized across the organization.
- Training programs exist.
- Engineering and management activities are integrated.

4. Level 4 – Managed

- Processes are measured and controlled quantitatively.
- Statistical techniques are applied.
- Focus on quality and productivity measurement.

5. Level 5 – Optimizing

- Continuous process improvement is the focus.
- Defect prevention, process optimization, and innovation are key.

Software Quality

Software Quality = The degree to which software meets customer requirements, expectations, and standards.

Key Dimensions of Quality

1. **Correctness** – Does it do what it should?
2. **Reliability** – Can it perform under expected conditions?
3. **Efficiency** – Optimal use of resources (CPU, memory).
4. **Integrity** – Security and protection from unauthorized access.
5. **Usability** – Easy to learn and use.
6. **Maintainability** – Easy to update, fix, or enhance.
7. **Portability** – Works in different environments.
8. **Testability** – Easy to test and validate.
9. **Flexibility** – Can adapt to changes.
10. **Reusability** – Components can be reused in other applications.

Role of Metrics and Measurement

Why Metrics?

- "You can't improve what you don't measure."
- Metrics provide **quantitative data** to assess software quality and process effectiveness.

Types of Metrics

1. Product Metrics

- Measure characteristics of the software product.
- Example: Size (LOC, Function Points), complexity (Cyclomatic Complexity), defect density.

2. Process Metrics

- Measure effectiveness of the software process.
- Example: Effort (person-hours), schedule variance, defect removal efficiency.

3. Project Metrics

- Focus on project management and progress.
- Example: Cost variance, productivity, team performance.

Examples of Important Metrics

- **LOC (Lines of Code):** Size measure.
- **Function Points:** Measures functionality delivered.
- **Cyclomatic Complexity (McCabe):** Measures control flow complexity.
- **Defect Density:** No. of defects per KLOC.
- **Mean Time to Failure (MTTF):** Reliability measure.

Benefits of Metrics

- Provides objective basis for decision-making.
- Helps track progress (time, cost, effort).
- Supports process improvement (CMMI, ISO 9001, Six Sigma).
- Improves quality by identifying weak areas.

Link Between CMM, Software Quality, and Metrics

- CMM Levels 4 & 5 rely heavily on quantitative measurement & metrics.
- **Software Quality** cannot be assured without **measurement** (you need metrics to evaluate correctness, reliability, etc.).
- **Metrics → Improve Quality → Achieve Higher Maturity in CMM.**

Software Quality:

Software Quality = *The degree to which a software product meets user requirements, expectations, and established standards.*

- It is not just about **working software**, but about **how well it works** under different conditions, and how easy it is to **maintain, use, and trust**.
- ISO 9126 / ISO 25010 defines software quality in terms of **quality attributes**.

Importance of Software Quality

- Reduces cost of rework and maintenance.
- Improves user satisfaction.
- Enhances reliability and trust.
- Helps meet industry standards (ISO 9001, CMMI, Six Sigma).
- Essential in critical domains (healthcare, aviation, defense, banking).

Dimensions / Factors of Software Quality

McCall's Quality Model (1977)

Defines three categories:

1. **Product Operation** → Correctness, Reliability, Efficiency, Integrity, Usability.
2. **Product Revision** → Maintainability, Flexibility, Testability.
3. **Product Transition** → Portability, Reusability, Interoperability.

ISO 25010 Software Quality Model

Defines **8 Quality Characteristics** (with sub-characteristics):

1. Functional Suitability

- Functional completeness
- Functional correctness
- Functional appropriateness

2. Performance Efficiency

- Time behavior (response time, throughput)
- Resource utilization
- Capacity

3. Compatibility

- Co-existence (works with other software on same environment)
- Interoperability (ability to exchange information with other systems)

4. Usability

- Learnability, Operability, Accessibility
- User interface aesthetics

5. Reliability

- Maturity, Availability, Fault tolerance, Recoverability

6. Security

- Confidentiality, Integrity, Non-repudiation, Authenticity

7. Maintainability

- Modularity, Reusability, Analyzability, Modifiability, Testability

8. Portability

- Adaptability, Installability, Replaceability

Software Quality Assurance (SQA)

SQA = A set of activities that ensures **processes and products conform to quality standards**.

Objectives of SQA

- Build confidence in software quality.
- Ensure compliance with standards.
- Prevent defects (not just detect).

SQA Activities

- Reviews (requirement review, code review).
- Audits (process audit, product audit).
- Testing (unit, integration, system, acceptance).
- Standards enforcement (ISO, IEEE, CMMI).

5. Software Quality Control (SQC)

- Subset of SQA.
- Involves **testing activities** to detect defects.
- Focused on product quality rather than process quality.

SQA = **Prevention** (process-oriented)

SQC = **Detection** (product-oriented)

6. Software Quality Metrics

Metrics help measure **quality quantitatively**.

Examples:

1. Reliability Metrics

- Mean Time to Failure (MTTF)
- Mean Time to Repair (MTTR)
- Availability = $MTTF / (MTTF + MTTR)$

2. Complexity Metrics

- Cyclomatic Complexity (McCabe's metric)
- Halstead's metrics

3. Size Metrics

- LOC (Lines of Code)
- Function Points

4. Defect Metrics

- Defect density = $\text{No. of defects} / \text{KLOC}$
- Defect removal efficiency (DRE)

Cost of Software Quality (CoSQ)

1. **Prevention Costs** → Cost of training, standards, reviews.

2. **Appraisal Costs** → Cost of inspection, testing.

3. **Failure Costs** →

- **Internal failure:** Defects found before release.
- **External failure:** Defects found after release (very costly).

Challenges in Ensuring Software Quality

- Changing customer requirements.
- Lack of communication between stakeholders.
- Trade-off between cost, time, and quality.
- Measuring non-functional attributes (usability, reliability).
- Distributed and agile development environments.

Requirements Analysis and Specification

A **requirement** is a description of what a system should do. It defines:

- *What to build* (functional requirements).
- *How the system should behave* (non-functional requirements).

Requirements act as a **bridge between customer needs and system design**.

Types of Requirements

1. Functional Requirements

- Define *what* the system should do.
- Example: “The system shall allow users to log in using a username and password.”

2. Non-functional Requirements

- Define *quality attributes* of the system.
- Example: Performance, security, reliability, usability, scalability.

3. Domain Requirements

- Derived from the application domain.
- Example: In a banking system, interest calculation rules.

Requirements Analysis Process

1. Feasibility Study

- Is the project economically, technically, and operationally feasible?

2. Requirement Elicitation (Gathering)

- Interviews, surveys, observation, brainstorming, prototyping.

3. Requirement Analysis

- Resolving conflicts, checking completeness, consistency, feasibility.

4. Requirement Specification

- Documenting requirements in a structured form (SRS document).

5. Requirement Validation

- Ensuring requirements are correct and reflect user needs.
- Techniques: Reviews, prototyping, test-case generation.

Software Requirements Specification (SRS)

An **SRS document** is the official contract between client and developer.

Contents of SRS (IEEE 830 standard):

1. Introduction

- Purpose, scope, definitions.

2. Overall description

- Product perspective, functions, constraints.

3. Functional requirements

4. Non-functional requirements

5. External interface requirements (UI, hardware, software).

6. Appendices & glossary

Qualities of a Good SRS:

- Correct, Complete, Unambiguous, Verifiable, Consistent, Modifiable, Traceable.

Software Project Planning:

Software Project Planning = Process of **defining scope, estimating effort, cost, resources, and scheduling activities** before starting development.

Goal: Deliver software on time, within budget, and with required quality.

Objectives of Project Planning

- Define project scope clearly.
- Identify resources (people, tools, hardware, software).
- Estimate cost, time, and effort.
- Identify risks and mitigation strategies.
- Create a project schedule.

Activities in Project Planning

1. Scope Definition

- Clearly define project boundaries, goals, and deliverables.

2. Effort Estimation

- Use models to estimate development effort.
- Common methods:
 - LOC (Lines of Code) estimation
 - Function Point Analysis (FPA)
 - COCOMO Model (Constructive Cost Model)

3. Resource Planning

- People: Skills required, team size.
- Tools: Development & testing environments.
- Hardware/Software resources.

4. Scheduling

- Break project into tasks.
- Tools:
 - **Gantt Chart** (time vs tasks)
 - **PERT/CPM Chart** (dependency analysis, critical path).

5. Risk Management

- Identify risks (technical, financial, schedule, requirement changes).
- Plan mitigation strategies.

6. Quality Planning

- Define quality objectives.
- Plan reviews, testing, and standards compliance.

Project Estimation Techniques

1. Lines of Code (LOC) Based Estimation

- Effort is estimated based on expected lines of source code.
- Limitation: Language-dependent, inaccurate for early stages.

2. Function Point Analysis (FPA)

- Measures software size by counting **functions delivered** (inputs, outputs, files, interfaces).
- Language-independent, better for early stages.

3. COCOMO Model

- Developed by Barry Boehm.
- Formula:

$$\text{Effort} = a \times (\text{KLOC})^b$$

where a and b depend on project type (organic, semi-detached, embedded).

- Estimates: Effort (person-months), Development time, Staffing.

Project Scheduling Tools

1. Gantt Chart

- Bar chart showing timeline of tasks.
- Simple visualization.

2. PERT/CPM (Program Evaluation and Review Technique / Critical Path Method)

- Graph-based representation.
- Identifies task dependencies and critical path (longest path in project).

Importance of Project Planning

- Prevents cost overruns and delays.
- Helps manage risks effectively.
- Provides clear roadmap for execution.
- Increases customer confidence.

Software Project Planning

Software Project Planning is the process of defining the scope, objectives, tasks, resources, schedule, and risks of a software development project in advance, to ensure that it can be completed within time, budget, and quality constraints.

Objectives of Project Planning

- Scope definition
- Cost, effort, time estimation
- Resource allocation
- Scheduling & milestones
- Risk identification & control
- Quality assurance

Steps in Project Planning

1. Scope Definition

- Boundaries, goals, exclusions
- Example: Online shopping system

2. Effort, Cost, and Time Estimation

- Techniques: LOC, Function Point Analysis, COCOMO
- Comparison of estimation models

3. Resource Planning

- Human resources, hardware, software
- Resource Allocation Matrix

4. Project Scheduling

- Work Breakdown Structure (WBS)
- Gantt charts
- PERT/CPM (critical path analysis)

5. Risk Management

- Types of risks (technical, schedule, cost, resource)
- Risk identification, analysis, mitigation, monitoring

6. Quality Planning

- Standards (ISO, CMMI)
- Metrics: defect density, reliability, usability

7. Documentation

- Project Plan Document (PPD) structure

Estimation Models

- LOC-based estimation
- Function Point Analysis (FPA)
- COCOMO (Basic, Intermediate, Detailed)
- Example calculations with sample projects

Project Scheduling Tools

- Gantt chart with figure
- PERT chart with critical path example
- Comparison between Gantt vs PERT
- Case study: Online Library Management scheduling

Importance of Project Planning

- Preventing overruns
- Customer satisfaction
- Resource optimization
- Lessons from real-world failed projects

Software Architecture (SA):

Software Architecture is the high-level structure of a software system, which defines its components, the relationships between them, and the principles guiding its design and evolution.

It acts as a blueprint for both the system and the project team, ensuring that the software meets functional requirements (what the system should do) and non-functional requirements (performance, security, scalability, reliability, etc.).

Objectives of Software Architecture

- Ensure quality attributes
- Support scalability, maintainability
- Enable stakeholder communication
- Reusability

Components of Software Architecture

- Modules
- Components & connectors
- Interfaces
- Data flow

Architectural Styles

1. Data-centered (repository, blackboard)
2. Data-flow (pipe and filter, batch sequential)
3. Call-and-return (main-subroutine, layered)
4. Object-oriented
5. Event-driven (publish-subscribe)
6. Client-server
7. Service-Oriented Architecture (SOA)
8. Microservices (modern evolution of SOA)

Design Patterns

- Creational (Singleton, Factory)
- Structural (Adapter, Composite)
- Behavioral (Observer, Strategy)
- Examples with diagrams

Quality Attributes

- Performance
- Reliability
- Usability
- Maintainability
- Security
- Scalability
- Trade-offs between attributes

Architectural Evaluation

- SAAM (Software Architecture Analysis Method)
- ATAM (Architecture Trade-off Analysis Method)
- Cost-benefit trade-offs

Documentation

- IEEE 1471 standard
- UML diagrams (component, deployment, sequence, package diagrams)
- Architecture Decision Records (ADR)

Importance of Software Architecture

- Managing complexity
- Supporting system evolution
- Enabling quality attributes
- Industry examples (Google, Amazon, Netflix)

Software Design and Implementation:

After requirements analysis and specification, the next step in the Software Development Life Cycle (SDLC) is Software Design, followed by Implementation (Coding).

- Software Design → The process of transforming requirements into a blueprint for building the system.
- Implementation → The process of writing code and realizing the design into a working software product.

Software Design:

Software Design is the process of defining the architecture, components, interfaces, and data for a system to satisfy specified requirements.

Objectives of Software Design

- To create a blueprint for implementation.
- To ensure system satisfies functional and non-functional requirements.
- To manage complexity through modularization.
- To enable reusability, maintainability, scalability, and reliability.

Levels of Design

1. High-Level Design (Architectural Design)

- Focuses on **system architecture**.
- Defines modules, components, subsystems, and their interactions.
- Example: Layered architecture, client-server, microservices.

2. Low-Level Design (Detailed Design)

- Focuses on **internal logic** of components.
- Describes algorithms, data structures, database schemas, and class diagrams.
- Example: Designing methods in a class, writing pseudo-code.

Design Principles

- **Modularity** → Divide system into independent modules.
- **Abstraction** → Focus on essential details, hide implementation.
- **Encapsulation** → Bind data and methods together.

- **Separation of Concerns** → Divide system based on responsibilities.
- **Coupling and Cohesion:**
 - Low coupling = less dependency between modules.
 - High cohesion = module focuses on a single responsibility.

Design Methods

- Structured Design (DFDs, ER diagrams, flowcharts).
- Object-Oriented Design (OOD) (UML diagrams, classes, objects, inheritance, polymorphism).
- Component-Based Design.

Software Design Tools

- UML Diagrams:
 - Class Diagram
 - Use Case Diagram
 - Sequence Diagram
 - Component Diagram
 - Deployment Diagram

Software Implementation:

Software Implementation (or Coding) is the stage where the design is translated into source code using a programming language.

Objectives of Implementation

- To convert design into working software.
- To follow **coding standards and guidelines**.
- To ensure correctness, efficiency, and maintainability.

- To support future **testing, debugging, and maintenance**.

Programming Paradigms

- Procedural Programming (C, Pascal).
- Object-Oriented Programming (Java, C++, Python).
- Functional Programming (Haskell, Scala, JavaScript functional parts).

Implementation Process

1. Select suitable programming language & environment.
2. Write code for modules.
3. Unit testing of individual modules.
4. Integrate modules together.
5. Perform system testing.

Tools for Implementation

- IDEs (Integrated Development Environments) → Eclipse, IntelliJ, PyCharm, Visual Studio.
- Version Control Systems → Git, GitHub, GitLab.
- Build Tools → Maven, Gradle, Make.
- Continuous Integration Tools → Jenkins, Travis CI.

Challenges in Implementation

- Requirement changes during coding.
- Bugs due to logic or integration issues.
- Performance bottlenecks.
- Managing large codebases.

Example

Case: Online Shopping System

- **Design Phase:**

- High-Level: 3-tier architecture (Presentation layer, Business logic, Database).
- Low-Level: Class design (User, Cart, Product, Order).

- **Implementation Phase:**

- Code classes in Java/Python.
- Connect to database (MySQL, MongoDB).
- Develop UI (React/Angular).

Software Testing and Reliability

Software Testing → A process of **evaluating a software system** to check whether it meets requirements and works correctly under different conditions.

Software Reliability → The probability that software will operate **without failure** for a specified time under given conditions.

Software Testing

Software Testing is a systematic process of **executing a program with the intent to find errors** and ensure the software meets **functional and non-functional requirements**.

Objectives of Testing

- To detect defects early.
- To ensure software meets user requirements.
- To improve quality, performance, security.
- To provide confidence before release.

Levels of Testing

Testing is not done in a single step. It is performed at different **levels**, starting from individual components up to the whole system. Each level has its **objectives, scope, and methods**.

The four main levels are:

1. **Unit Testing**
2. **Integration Testing**
3. **System Testing**
4. **Acceptance Testing**

1. Unit Testing

Unit Testing checks the **smallest testable parts** of software, usually functions, classes, or modules, to verify they work as expected.

Purpose

- Validate correctness of **individual units**.
- Detect errors early in development.
- Ensure code behaves as per design.

Who Performs?

- Developers (often with automated tools).

Techniques

- **White-box testing** → path coverage, condition coverage.
- Use of **stubs** and **drivers** when modules depend on others.

Tools

- JUnit (Java), NUnit (.NET), pytest (Python).

Example: Testing a function that calculates the interest in a banking app.

2. Integration Testing

Integration Testing ensures that **modules or units work together** when combined.

Purpose

- Detect interface defects (mismatched data types, incorrect API calls).
- Ensure communication between modules is correct.

Who Performs?

- Developers or testers.

Approaches

1. **Top-Down Integration** → Start with main control module, integrate downwards using *stubs*.
2. **Bottom-Up Integration** → Start with lower modules, combine upwards using *drivers*.
3. **Big-Bang Integration** → All modules integrated at once (risk of chaos).
4. **Sandwich/Hybrid Integration** → Combination of top-down and bottom-up.

Example: Checking whether the login module passes correct user data to the dashboard module.

3. System Testing

System Testing tests the **entire application** as a whole, validating both functional and non-functional requirements.

Purpose

- Verify that the system meets its specifications.
- Test in an environment similar to production.

Who Performs?

- Independent testing team (QA).

Types

- **Functional Testing** → Requirements validation.
- **Performance Testing** → Load, stress, scalability.
- **Security Testing** → Vulnerability checks, penetration tests.
- **Usability Testing** → User-friendliness.
- **Compatibility Testing** → Different browsers, devices, OS.

Example: Testing an e-commerce website with complete order processing (search, cart, payment, confirmation).

4. Acceptance Testing

Definition

Acceptance Testing validates the software **against user/business requirements** and determines if it is ready for release.

Purpose

- Ensure the software solves the customer's problem.
- Provide confidence before deployment.

Who Performs?

- End-users, clients, or QA team on behalf of the client.

Types

1. **Alpha Testing** → Done at developer's site, with limited user involvement.
2. **Beta Testing** → Done at customer's site, with real-world users.
3. **User Acceptance Testing (UAT)** → Performed by end-users.

Example: A bank verifies its new online transaction system before releasing it to customers.

Level	Scope	Focus	Performed By
Unit Testing	Single module	Correctness of code	Developers
Integration Test	Interfaces	Module interaction	Dev/QA
System Testing	Whole system	Requirements	QA Team
Acceptance Test	Real use case	Business needs	Users/Clients

Types of Software Testing

Software Testing types are generally divided into **Functional Testing** and **Non-Functional Testing**, but there are also **Maintenance Testing** and **Specialized Testing** categories.

1. Functional Testing

Functional testing checks **what the system does** — i.e., does the software behave as per requirements?

Main Types

a. Unit Testing

- Smallest testable part (functions, methods).
- Ensures correctness of individual components.

b. Integration Testing

- Verifies interaction between modules.
- Example: Login module passing data to dashboard module.

c. System Testing

- Validates the complete system as per requirements.
- Example: An e-commerce site from product search to payment.

d. Regression Testing

- Ensures new code does not break old functionality.
- Done after bug fixes or enhancements.

e. Sanity Testing

- Quick test of specific functionality after small changes.

f. Smoke Testing

- Initial "build verification testing".
- Checks if the basic system runs without crashing.

g. User Acceptance Testing (UAT)

- Performed by end-users.
- Confirms system meets **business needs**.

2. Non-Functional Testing

Non-functional testing checks **how well the system works** — i.e., performance, usability, reliability, etc.

Main Types

a. Performance Testing

- Evaluates speed, response time, and resource usage.
- **Load Testing** → System under expected load.
- **Stress Testing** → Extreme load beyond limits.
- **Soak (Endurance) Testing** → Long-duration usage.

b. Security Testing

- Identifies system vulnerabilities.
- Includes penetration testing, ethical hacking.
- Example: Checking if password data is encrypted.

c. Usability Testing

- Ensures software is **user-friendly**.
- Example: Simple navigation, readable fonts.

d. Compatibility Testing

- Runs on multiple browsers, devices, OS.
- Example: Website works on Chrome, Firefox, Safari.

e. Scalability Testing

- Measures ability to handle growing workload.
- Example: E-commerce site handling Black Friday traffic.

f. Reliability Testing

- Ensures software works without failure for a period.

g. Portability Testing

- Checks if software runs on different environments.
- Example: Mobile app running on Android & iOS.

3. Maintenance Testing

Done during **software maintenance** (after release).

- **Regression Testing** → Ensures updates don't break old features.
- **Re-testing (Confirmation Testing)** → Ensures fixed bug really works.

Specialized Testing

Advanced/specific testing types:

1. Alpha Testing

- At developer's site, internal testing with limited users.

2. Beta Testing

- Real-world testing by actual customers before official release.

3. Ad-hoc Testing

- Random, unstructured testing without documentation.

4. Exploratory Testing

- Tester explores software while learning it.

5. Monkey Testing

- Random inputs, stress testing software unexpectedly.

6. Mutation Testing

- Injecting small changes (mutants) in code to check if test cases detect them.

7. A/B Testing

- Comparing two versions (common in web apps, marketing).

Software Testing Techniques

Testing techniques are structured **methods used to design test cases** and ensure that the system is tested effectively.

They are broadly divided into:

1. **Black-Box Testing Techniques**
2. **White-Box Testing Techniques**
3. **Gray-Box Testing Techniques**

1. Black-Box Testing Techniques

Black-box testing focuses on the **functionality** of the system without looking at internal code.

The tester only cares about **inputs and expected outputs**.

1.1 Equivalence Partitioning (EP)

- Divide input data into **equivalence classes** where all values should behave similarly.
- Choose one representative from each class.

Example:

If valid age is 18–60 →

- Class 1: below 18 (invalid).
- Class 2: 18–60 (valid).
- Class 3: above 60 (invalid).

1.2 Boundary Value Analysis (BVA)

- Errors often occur at boundaries.
- Test values just **below, at, and above boundaries**.

Example:

Age input 18–60 → test **17, 18, 60, 61**.

1.3 Decision Table Testing

- Used when system behavior depends on **combinations of conditions**.
- Represents inputs/conditions and outputs/actions in a **table**.

Example:

Loan approval depends on **salary & credit score**.

Salary > 50k	Credit Good	Action
Yes	Yes	Approve
Yes	No	Review
No	Yes	Review
No	No	Reject

1.4 State Transition Testing

- Used when system behavior changes with **states and events**.
- Test all possible **state changes**.

Example:

ATM → States: Idle → PIN Entered → Transaction → Idle.

1.5 Use Case Testing

- Based on real-world **user scenarios**.
- Each **use case = test case**.

Example: Shopping cart → Add product → Checkout → Payment → Confirmation.

2. White-Box Testing Techniques

White-box testing requires **knowledge of internal code logic**. It ensures that **all paths, conditions, and loops** are tested.

2.1 Statement Coverage

- Every **line of code** must be executed at least once.

2.2 Branch Coverage (Decision Coverage)

- Every **true/false branch** of decision points must be tested.

2.3 Condition Coverage

- Each **boolean condition** in a decision must be tested for both true/false.

2.4 Path Coverage

- Ensures **all possible execution paths** are tested.
- Very thorough but expensive.

2.5 Loop Testing

- Special focus on loops → test **0, 1, many iterations**.

3. Gray-Box Testing Techniques

Combination of **black-box** and **white-box**. Tester has **partial knowledge** of the system internals.

Examples

- Testing a web app with knowledge of DB schema.
- API testing with knowledge of request/response formats.
- Security testing with partial knowledge of source code.

Software Reliability

Software Reliability is the probability that software will function correctly without failure under specified conditions for a given time period.

Reliability = Correctness + Robustness + Availability.

Factors Affecting Reliability

- Complexity of software (more LOC → more bugs).
- Development process quality.
- Testing effectiveness.
- Operating environment (hardware, users, network).
- Fault tolerance mechanisms.

Reliability Metrics

Reliability is one of the most important non-functional requirements of software. Since software never “wears out” like hardware, we need mathematical and statistical measures to quantify how reliable a system is. These measures are called Reliability Metrics.

They help in:

- Predicting software performance.
- Estimating downtime.
- Comparing different systems.
- Making design and maintenance decisions.

1. MTTF (Mean Time to Failure)

- Definition: The average time a system runs before it fails.
- Formula:

$$\text{MTTF} = \text{Total operational time} / \text{Number of failures}$$

- Example:

If software runs for 1,000 hours and fails 5 times:

$$MTTF=1000/5=200$$

- Use: For non-repairable systems (embedded software, appliances).

2. MTTR (Mean Time to Repair)

- The average time taken to fix a failure and restore the system.
- Formula:

$$MTTR=\text{Total repair time}/\text{Number of repairs}$$

- Example:

If 5 failures took 10, 8, 12, 6, and 14 hours to fix:

$$MTTR=(10+8+12+6+14)/5=10 \text{ hours}$$

- Use: Helps estimate system downtime.

3. MTBF (Mean Time Between Failures)

- The expected time between two successive failures.
- Formula:

$$MTBF=MTTF+MTTR$$

- Example:

If MTTF = 200 hours and MTTR = 10 hours:

$$MTBF=200+10=210 \text{ hours}$$

- Use: Commonly used in reliable systems that can be repaired.

4. Failure Rate (λ)

- Definition: The frequency of system failures per unit time.
- Formula:

$$\lambda = 1/MTTF$$

- Example:
If MTTF = 200 hours:

$$\lambda = 1/200 = 0.005 \text{ failures/hour}$$

- Use: Indicates how often a system fails.

5. Reliability Function (R(t))

- Probability that the system will operate without failure over time t.
- Formula:

$$R(t) = e^{-\lambda t}$$

- Example:
If $\lambda = 0.005$ failures/hour, probability of running for 100 hours:

$$R(100) = e^{-0.005 \times 100} = e^{-0.5} \approx 0.6065$$

→ The system has 60.65% chance of surviving 100 hours without failure.

6. Availability (A)

- Probability that a system is operational at any given time.
- Formula:

$$A = \text{MTTF} / (\text{MTTF} + \text{MTTR})$$

- Example:
If MTTF = 200 hours and MTTR = 10 hours:

$$A = 200 / (200 + 10) = 0.952 \text{ or } 95.2\%$$

- Use: Critical for 24/7 services (banking, cloud, telecom).

7. Defect Density

- Number of defects per unit size of software (e.g., per KLOC).
- Formula:

$$\text{Defect Density} = \text{Total defects} / \text{KLOC (thousand lines of code)}$$

- Example:

If 40 defects are found in 20 KLOC:

$$\text{Defect Density} = 40/20 = 2 \text{ defects/KLOC}$$

- Use: Compares code quality between modules/projects.

8. Fault Density

- Definition: Number of faults per unit of test cases executed.
- Formula:

$$\text{Fault Density} = \text{Faults detected} / \text{Test cases executed}$$

9. Service Reliability Metrics

- Downtime per year = Hours system is unavailable.
- Uptime percentage (commonly used in SLAs):

$$\text{Uptime} = [(\text{Total time} - \text{Downtime}) / \text{Total time}] \times 100$$

- 99% uptime → 3.65 days downtime per year.
- 99.9% uptime → 8.76 hours downtime per year.
- 99.99% uptime → 52.6 minutes downtime per year.

Classification of Reliability Metrics

1. Time-based metrics → MTTF, MTTR, MTBF, λ , R(t).
2. Defect-based metrics → Defect density, Fault density.
3. Availability-based metrics → Availability (A), Uptime percentage.
4. Customer-based metrics → SLA compliance, Mean User Downtime.

Applications of Reliability Metrics

- Designing reliable systems (aviation, healthcare, finance).
- Comparing vendors (based on SLA uptime).
- Evaluating testing effectiveness.
- Predicting failures in maintenance planning.
- Supporting reliability growth models (Musa, Goel-Okumoto).

